

Atividade_07 – Interrupções externas

Título: Utilizando interrupções externas para medir frequência de eventos

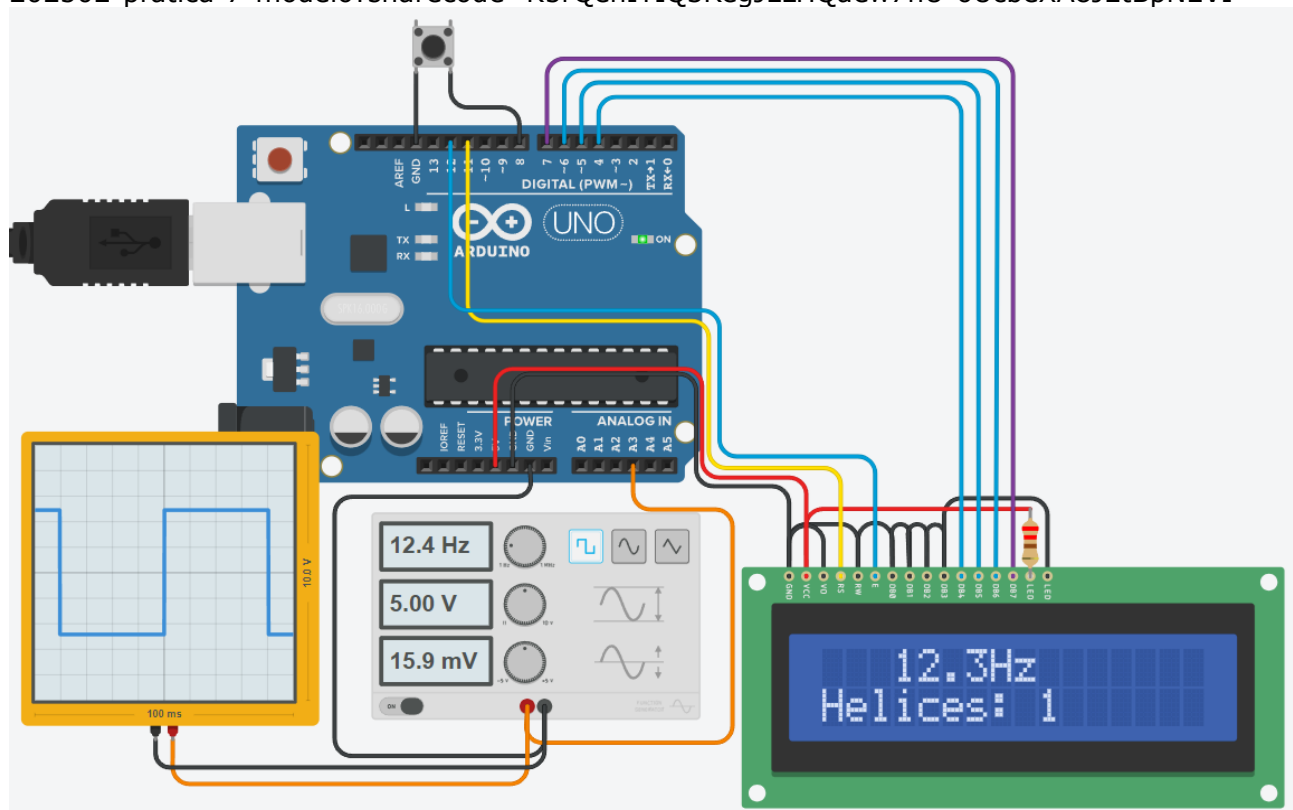
Objetivos: O objetivo é construir um tacômetro, aparelho capaz de medir a velocidade de rotação de objetos. Normalmente tal aparelho utiliza a leitura do reflexo de um feixe laser para calcular a rotação do eixo sob medida.



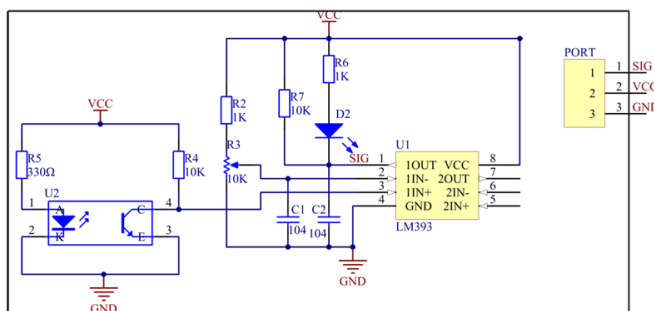
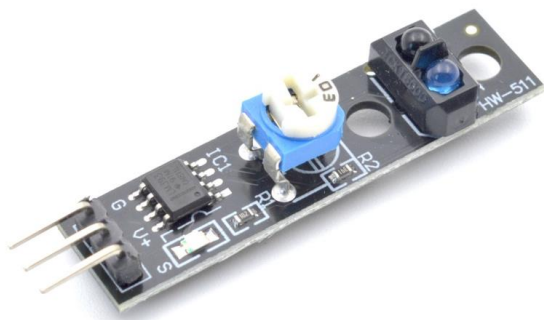
Nesta prática utilizaremos o **Tinkercad** para simular um circuito simples usando o microcontrolador Atmega328, utilizado nas placas Arduino UNO.

1. Procedimentos:

1. Copie o projeto fornecido em <https://www.tinkercad.com/things/fk6qkVQ5vZn-external-interrupt-cf-202502-pratica-7-modelo?sharecode=K5rQehITIQ3RegJLLHQAew7nU-oUcbeXA8JEtBpNEVI>



No circuito físico, utilizamos uma placa com um transmissor e receptor infravermelho que chaveia mediante o reflexo infravermelho recebido. A placa é a HW-511, conhecida como "sensor segue linha". Nela temos o conjunto transmissor-receptor TCRT5000 e um comparador analógico LM393 para fazer a histeria do sinal.



Vamos considerar o caso base de uma única hélice, caso em que um pulso representa uma única revolução. Como a atualização do display ocorre a cada 100ms, podemos contar quantos pulsos ocorreram neste intervalo e multiplicar por 10 para obter as revoluções por segundo. No entanto, se a frequência for muito baixa, a precisão e acurácia da medida fica muito prejudicada.

Utilize o botão fornecido para permitir a configuração do número de hélices. A cada clique o número é incrementado até atingir 9 e voltar a 1. Quando o número de hélices é maior que 1, basta dividir o valor obtido anteriormente para estimar a frequência de rotação do eixo.

A taxa de atualização do LCD deve ser de **10Hz**, ou seja, você deve atualizar o display LCD somente **a cada 100ms**.

DICA: Utilize `dtostrl()` para converter suas variáveis float para string.

RESTRIÇÃO DE PROJETO: O circuito utilizado no projeto deve ser o fornecido, podendo as mudanças ocorrerem somente em relação ao software. Não devem haver delays (espera ociosa), exceto para o caso da simulação rodar mais rapidamente (ou rotinas do display que já possuem tais delays). Não utilizar bibliotecas externas, como o LiquidCristal. Deve-se utilizar o código passado em aula. Este item é obrigatório e zera a avaliação caso não seja atendido.

Atenção: Ao utilizar main() ao invés de setup() e loop(), lembre-se de chamar a função init() para a inicialização do timer e correto funcionamento do millis().

Atenção: Não utilizar `digitalRead()`. Este item é obrigatório e zera a avaliação caso não seja atendido.

4. Cole o código fonte do microcontrolador ao final deste arquivo.

OBRIGATÓRIO PARA AVALIAÇÃO: torne seu projeto compartilhado por link (não público) e cole o link para ele aqui:

<https://www.tinkercad.com/things/bD7rcXDs8WU-copy-of-external-interrupt-cf-202601-pratica-7-modelo?sharecode=SFFyqDlyv0uPYQT5kZmudXUSO9rIM4xQpkQ3WWvhrkQ>

ATENÇÃO: Documente seu código. Cada linha/bloco deve deixar explícito o seu papel.

RÚBRICA:

Lógica da programação e funcionamento (desde que atendendo às restrições de projeto): 100%

Valor desta atividade na média: 0.8

6. Notas extras:

6.1 Adicionar um segundo botão que alterna entre Hz e RPM, ou seja, revoluções por segundo e revoluções por minuto (trocar no LCD os dizeres 'Hz' por 'RPM' além de fazer os cálculos): 0.1

```
#include <stdio.h>
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdlib.h>

// LCD
#define LCD_DADOS PORTD    // PD4 até PD7
#define LCD_CTRL  PORTB

#define RS PB3
#define EN PB4

// Botão
#define BOTAO PB0          // D8

// Pulso de enable do LCD
#define pulsoEnable()      \
    _delay_us(1);          \
    LCD_CTRL |= (1 << EN);  \
    _delay_us(1);          \
    LCD_CTRL &= ~(1 << EN); \
    _delay_us(45);

// Protótipos
void lcd_cmd(unsigned char valor, char tipo);
void lcd_init();
void lcd_print(const char *texto);

// Variáveis globais
volatile unsigned long ultimoPulso = 0;
volatile unsigned long intervaloPulso = 0;
volatile unsigned int contadorPulsos = 0;

volatile uint8_t estadoAnteriorPC3 = 0;
```

```
unsigned long tempoLCD = 0;
unsigned long tempoBotao = 0;
```

```
int helices = 1;
uint8_t estadoAnteriorBotao = 0;
```

```
// Envia comando ou caractere para o LCD
```

```
void lcd_cmd(unsigned char valor, char tipo) {

    if (tipo == 0)
        LCD_CTRL &= ~(1 << RS); // comando
    else
        LCD_CTRL |= (1 << RS); // dado

    // nibble alto
    LCD_DADOS = (LCD_DADOS & 0x0F) | (valor & 0xF0);
    pulsoEnable();

    // nibble baixo
    LCD_DADOS = (LCD_DADOS & 0x0F) | (valor << 4);
    pulsoEnable();

    if ((tipo == 0) && (valor < 4))
        _delay_ms(2);
}
```

```
// Inicialização do LCD 4 bits
```

```
void lcd_init() {

    LCD_CTRL &= ~(1 << RS);
    LCD_CTRL &= ~(1 << EN);

    _delay_ms(20);

    lcd_cmd(0x33, 0);
    _delay_ms(5);

    lcd_cmd(0x32, 0);

    lcd_cmd(0x28, 0); // 4 bits / 2 linhas
    lcd_cmd(0x0C, 0); // display ligado
    lcd_cmd(0x01, 0); // limpa tela

    _delay_ms(2);
}
```

```
// Escreve string no LCD
```

```
void lcd_print(const char *texto) {
    while (*texto) {
        lcd_cmd(*texto++, 1);
    }
}
```

```

// Configuração inicial

void setup() {

    // LCD como saída
    DDRB |= (1 << RS) | (1 << EN);
    DDRD |= 0xF0;

    // Botão como entrada com pull-up
    DDRB &= ~(1 << BOTAO);
    PORTB |= (1 << BOTAO);

    // A3 (PC3) como entrada
    DDRC &= ~(1 << PC3);

    lcd_init();

    // Interrupção no PC3
    PCICR |= (1 << PCIE1);
    PCMSK1 |= (1 << PCINT11);

    sei();
}

// Loop principal

void loop() {

    unsigned long agora = millis();

    // Leitura do botão
    uint8_t estadoBotao = (PINB & (1 << BOTAO));

    // botão pressionado
    if (!estadoBotao &&
        estadoAnteriorBotao &&
        (agora - tempoBotao > 200)) {

        helices++;

        if (helices > 9)
            helices = 1;

        tempoBotao = agora;
    }

    estadoAnteriorBotao = estadoBotao;

    // Atualiza LCD
    if (agora - tempoLCD >= 100) {

        tempoLCD = agora;

        cli();

        unsigned long intervalo = intervaloPulso;

```

```

    unsigned int pulsos = contadorPulsos;
    unsigned long semPulso = agora - ultimoPulso;

    contadorPulsos = 0;

    sei();

    float freqTotal = 0;

    // eixo parado
    if (semPulso > 1000) {

        freqTotal = 0;

    } else {

        // baixa frequência
        if (intervalo > 10)
            freqTotal = 1000.0 / (float)intervalo;

        // alta frequência
        else
            freqTotal = pulsos * 10.0;
    }

    float freqFinal = freqTotal / helices;

    // Texto LCD
    char textoFreq[10];
    dtostrf(freqFinal, 4, 1, textoFreq);

    char linha1[17];
    char linha2[17];

    sprintf(linha1, "%s Hz      ", textoFreq);
    sprintf(linha2, "Helices: %d  ", helices);

    lcd_cmd(0x80, 0);
    lcd_print(linha1);

    lcd_cmd(0xC0, 0);
    lcd_print(linha2);
}

delay(1);
}

// Interrupção do PC3 (A3)
ISR(PCINT1_vect) {

    uint8_t estadoAtual = (PINC & (1 << PC3));

    // detecta borda de subida
    if (estadoAtual && !estadoAnteriorPC3) {

        unsigned long tempoAtual = millis();
    }
}

```

```
    intervaloPulso = tempoAtual - ultimoPulso;  
    ultimoPulso = tempoAtual;  
  
    contadorPulsos++;  
}  
  
estadoAnteriorPC3 = estadoAtual;  
}
```